第15章源码阅读

由正式源文件生成的静态阅读副本。返回正文 · 运行说明

---

src/main/java/org/example/jvminternals/ch15/DynamicJourney.java

```java
package org.example.jvminternals.ch15;

import java.lang.classfile.ClassFile;
import java.lang.constant.ClassDesc;
import java.lang.constant.ConstantDescs;
import java.lang.constant.DynamicCallSiteDesc;
import java.lang.constant.DynamicConstantDesc;
import java.lang.constant.MethodTypeDesc;
import java.lang.invoke.CallSite;
import java.lang.invoke.MethodHandle;
import java.lang.invoke.MethodHandles;
import java.lang.invoke.MethodType;
import java.lang.invoke.MutableCallSite;
import java.lang.reflect.Proxy;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.function.IntUnaryOperator;

/** 少量调用展示链接与类型适配，不统计首次或稳定调用耗时。 */
public final class DynamicJourney {
    private static MutableCallSite linkedSite;

    private DynamicJourney() {
    }

    /** 公开接口使代理可合法转交到目标对象。 */
    public interface Greeting {
        String greet(String name);
    }

    /** 反射和代理共用的实际目标。 */
    public static final class Greeter implements Greeting {
        @Override
        public String greet(String name) {
            return "你好，" + name;
        }
    }
```

```
    /** 动态调用引导方法：该输出只标记本轮链接，不假定并发时仅执行一次。 */
    public static CallSite bootstrapCall(MethodHandles.Lookup caller, String name,
MethodType type) {
        System.out.println("引导=调用点；名称=" + name + "；调用类=" +
caller.lookupClass().getName());
        linkedSite = new MutableCallSite(MethodHandles.constant(String.class, "初始目
标").asType(type));
        return linkedSite;
    }

    /** 动态常量引导方法：返回供常量池解析缓存的值。 */
    public static Object bootstrapConstant(MethodHandles.Lookup caller, String name,
Class<?> type) {
        System.out.println("引导=常量；名称=" + name + "；类型=" + type.getName());
        return "已解析值";
    }

    /** 依次展示句柄、Lambda / 拼接、反射 / 代理和真实类文件动态常量。 */
    public static void main(String[] args) throws Throwable {
        MethodHandles.Lookup lookup = MethodHandles.lookup();
        MethodHandle add = lookup.findStatic(DynamicJourney.class, "add",
MethodType.methodType(int.class, int.class, int.class));
        MethodHandle bound = MethodHandles.insertArguments(add, 0, 7);
        int exact = (int) bound.invokeExact(5);
        long adapted = (long) bound.invoke((byte) 5);
        System.out.println("句柄=精确" + exact + "；适配=" + adapted + "；类型=" +
bound.type());
        int bias = 7;
        IntUnaryOperator operation = value -> value + bias;
        System.out.println("Lambda首次=" + operation.applyAsInt(5));
        System.out.println("Lambda后续=" + operation.applyAsInt(6) + "；隐藏=" +
operation.getClass().isHidden());
        Greeter target = new Greeter();
        var method = Greeting.class.getMethod("greet", String.class);
        System.out.println("反射=" + method.invoke(target, "读者"));
        Greeting proxy = (Greeting)
Proxy.newProxyInstance(DynamicJourney.class.getClassLoader(), new Class<?>[]
{Greeting.class},
                (instance, called, arguments) -> called.invoke(target, arguments));
        System.out.println("代理=" + proxy.greet("读者"));
        Class<?> generated = lookup.defineClass(generateClass());
        MethodHandle dynamic = lookup.findStatic(generated, "dynamic",
MethodType.methodType(String.class));
        MethodHandle constant = lookup.findStatic(generated, "constant",
MethodType.methodType(String.class));
        System.out.println("阶段=动态调用首次；值=" + (String) dynamic.invokeExact());
        System.out.println("阶段=动态调用后续；值=" + (String) dynamic.invokeExact());
        // 更换目标保持方法类型，显式同步协议也供多线程读者理解。
        linkedSite.setTarget(MethodHandles.constant(String.class, "更新目标"));
        MutableCallSite.syncAll(new MutableCallSite[]{linkedSite});
        System.out.println("阶段=动态调用更新；值=" + (String) dynamic.invokeExact());
```

```
        System.out.println("阶段=常量首次；值=" + (String) constant.invokeExact());
        System.out.println("阶段=常量后续；值=" + (String) constant.invokeExact());
        System.out.println("阶段=正常结束");
    }

    private static int add(int left, int right) {
        return left + right;
    }

    private static byte[] generateClass() throws java.io.IOException {
        ClassDesc owner = ClassDesc.of(DynamicJourney.class.getName());
        var callBootstrap = ConstantDescs.ofCallsiteBootstrap(owner, "bootstrapCall",
ConstantDescs.CD_CallSite);
        var constantBootstrap = ConstantDescs.ofConstantBootstrap(owner,
"bootstrapConstant", ConstantDescs.CD_Object);
        var dynamic = DynamicCallSiteDesc.of(callBootstrap, "message",
MethodTypeDesc.of(ConstantDescs.CD_String));
        var constant = DynamicConstantDesc.ofNamed(constantBootstrap, "message",
ConstantDescs.CD_String);
        byte[] bytes =
ClassFile.of().build(ClassDesc.of("org.example.jvminternals.ch15.GeneratedLinkage"),
builder -> builder
                .withFlags(ClassFile.ACC_PUBLIC | ClassFile.ACC_SUPER)
                .withMethodBody("dynamic", MethodTypeDesc.of(ConstantDescs.CD_String),
ClassFile.ACC_PUBLIC | ClassFile.ACC_STATIC,
                        code -> code.invokedynamic(dynamic).areturn())
                .withMethodBody("constant", MethodTypeDesc.of(ConstantDescs.CD_String),
ClassFile.ACC_PUBLIC | ClassFile.ACC_STATIC,
                        code -> code.ldc(constant).areturn()));
        // 仅生成本章教学类文件；写到构建输出供javap读取，不生成测试夹具。
        Path output = Path.of("chapters/ch15-dynamic-
invocation/target/generated/GeneratedLinkage.class");
        Files.createDirectories(output.getParent());
        Files.write(output, bytes);
        return bytes;
    }
}
```